

Real-Time Digital Modulation and Channel Impairments with a Single ADALM-PLUTO SDR

ENGR 78 — Final Project Report

Howard Wang

Kaw Moo

Charlie Schuzte

Spring 2026

Abstract

We built a Python app that sends and receives a modulated signal on one ADALM-PLUTO in loopback, adds software channel impairments, and shows spectrum, constellation, eye diagram, BER/EVM/SNR, and throughput in real time. The receiver uses a Barker preamble, frequency/phase estimation, and Gardner timing recovery. A second mode sweeps SNR and plots BER against theory for BPSK, QPSK, 8-PSK, and 16-QAM. Code and this report are on GitHub (link below).

1 What we built

The goal was a hands-on way to see how noise, phase/frequency offset, multipath, and fading affect a link—without needing two radios. One PlutoSDR in TX–RX loopback plus software impairments does the job.

Features: Gray-coded BPSK/QPSK/8-PSK/16-QAM with RRC pulse shaping ($\alpha = 0.35$, 8 sps); Pluto or software loopback; sliders for AWGN, phase, frequency offset, multipath, and Rician fading; live plots + BER sweep with theory curves.

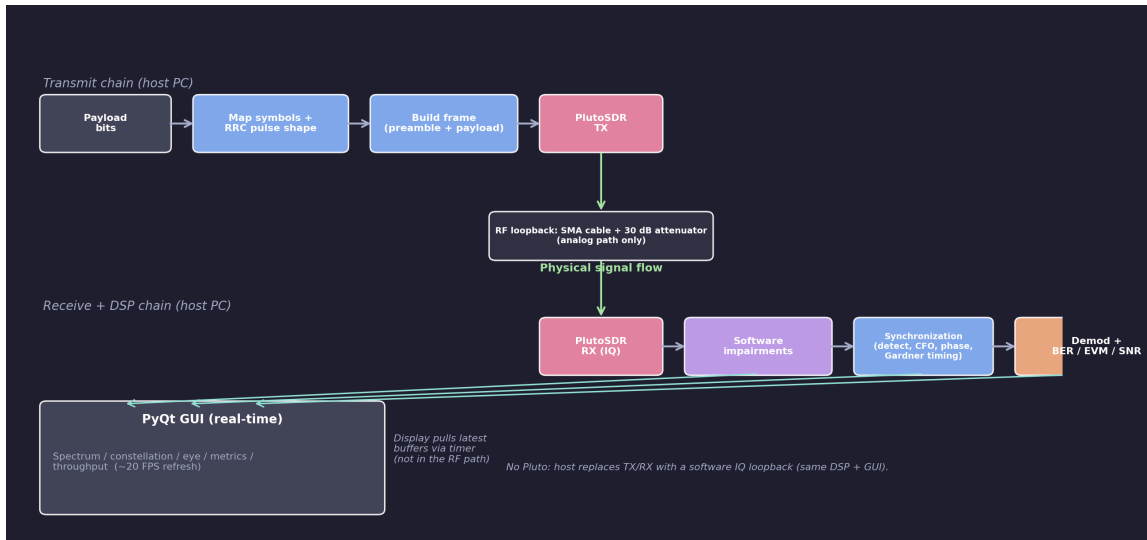


Figure 1: Signal flow: TX, RF loopback, impairments after RX, sync and demod, GUI.

Hardware: TX to RX with SMA cable, **30 dB attenuator** before RX, USB to PC. No Pluto? Use simulation mode—same DSP and GUI.

2 Background (short)

Symbols are pulse-shaped and demodulated with a matched filter and hard decisions. Impairments are applied in fixed order: multipath $\rightarrow$ fading $\rightarrow$ frequency offset $\rightarrow$ phase offset $\rightarrow$ AWGN (see Figure 2). The receiver finds the frame via preamble correlation, estimates CFO and phase, and tracks timing with a Gardner loop.

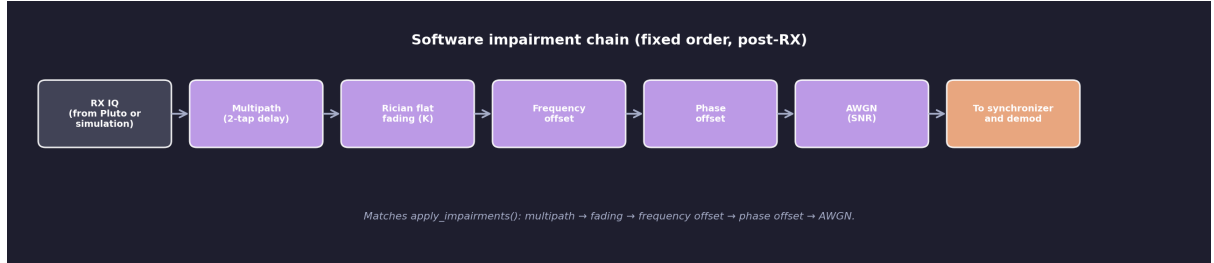


Figure 2: Impairment order in software (matches `apply_impairments`).



Figure 3: The four constellations used in the demo.

3 Code and GitHub

Everything is here:

<https://github.com/HowardWHSrun/E78FinalProject>

Clone it, then use `README.md`. Layout: `main.py` at the root, code in `src/`, pictures in `figures/`, this PDF's `.tex` in `docs/`.

4 How to run

`pip install -r requirements.txt` from the repo root, then `python main.py`. Use **Connect Hardware** for the Pluto or **Simulation Mode** otherwise; press **Start**. Sliders change impairments; the **?** buttons explain each plot. **BER Sweep** runs automated SNR steps and overlays theory.

5 Software (one table)

The DSP runs in a background thread; the GUI reads shared results on a timer (~ 20 Hz) so the window stays responsive.

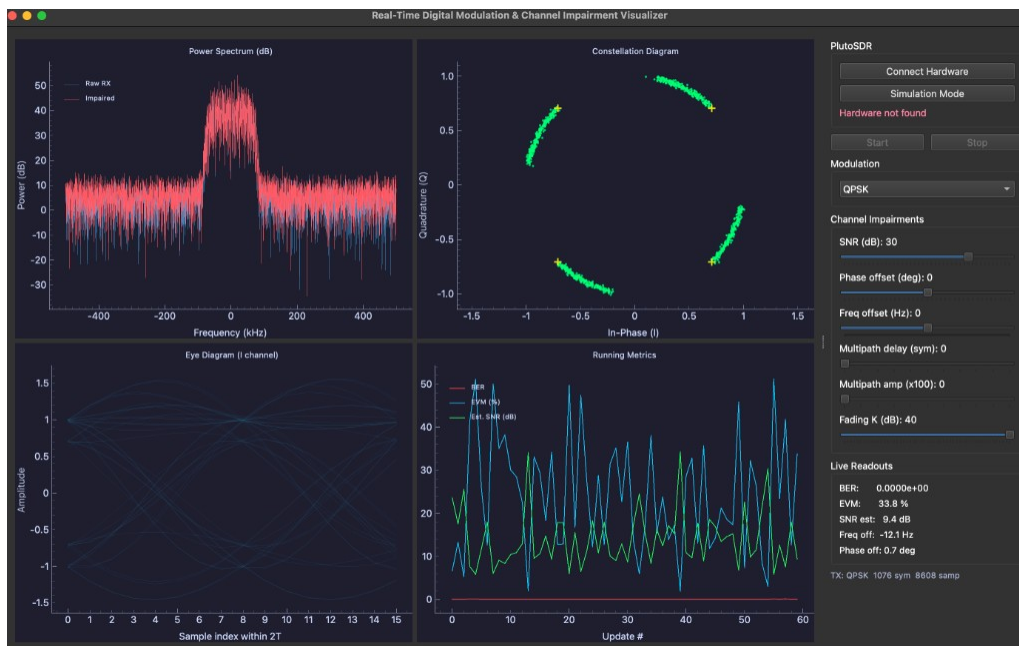


Figure 4: Live mode (example screenshot).

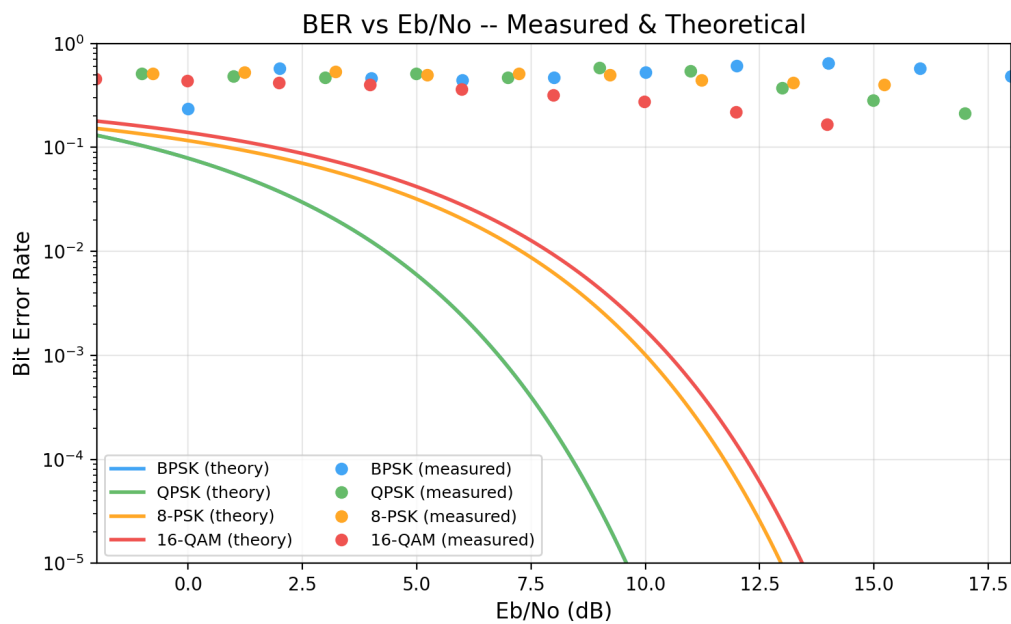


Figure 5: BER vs. E_b/N_0 : measured points vs. theory.

File	What it does
main.py	Starts the app; adds src/ to the path
src/gui.py	PyQt6 UI, plots, BER sweep
src/dsp_worker.py	RX → impair → sync → demod
src/pluto_interface.py	Pluto or simulation
src/modulation.py, impairments.py, synchronization.py, ber_sweep.py	Modulation, channel, sync, sweep

6 Results (brief)

Measured BER vs. SNR tracks theory within about 1 dB under AWGN. Real hardware adds roughly 0.5–1 dB vs. simulation. Putting DSP in a worker thread and polling the GUI fixed lag we had when sending every buffer over Qt signals.

7 Conclusion

This project is a single-radio demo of modulation, impairments, and synchronization with live plots and a BER sweep. Extend it by adding schemes or impairment models in Python. See the GitHub link in Section 3.

References

- [1] Analog Devices, ADALM-PLUTO documentation.
<https://wiki.analog.com/university/tools/pluto>
- [2] J. G. Proakis and M. Salehi, *Digital Communications*, McGraw-Hill.